

Manual Técnico de Desarrollo

Proyecto: Sistema de Trazabilidad Láctea — Gemelo Digital

Versión: 1.0 | Fecha: Junio 2026

Contenido

- Guía de configuración del entorno de desarrollo
 - Requisitos
 - Backend (.NET 8)
 - Frontend web (Next.js 16)
 - Frontend mobile (Expo 54)
 - Ejecución con Expo Go
- Estructura de directorios
 - Monorepo — ProyectoIntegrador
 - Backend — backend_integrador
- Arquitectura del código fuente
- Convenciones de código y estándares
- Descripción de módulos y componentes
 - Paquetes compartidos
 - Frontend mobile
 - Frontend web
 - Backend — capas y servicios
- APIs y endpoints documentados
 - Endpoints públicos
 - Endpoints autenticados
 - Endpoints del gemelo digital
 - Formato de respuesta
- Guía de testing
 - Tests unitarios (backend)
 - Tests de integración (backend)
 - Tests mobile
 - Tests web
- Proceso de CI/CD
- Gestión de versiones (Git Flow)
- Contribución al proyecto

1. Guía de configuración del entorno de desarrollo

1.1 Requisitos

Herramienta	Versión mínima	Uso
Git	2.40+	Control de versiones
.NET SDK	8.0	Compilación del backend
Node.js	22 LTS	Frontend web y mobile
PostgreSQL	16	Base de datos
Expo CLI	—	Build mobile (<code>npx expo</code>)
Docker Desktop	—	Opcional (backend containerizado)

1.2 Backend (.NET 8)

Paso 1 — Clonar

```
git clone https://github.com/anomalyco/backend_integrador.git
cd backend_integrador
```

Paso 2 — Crear base de datos

```
psql -U postgres -c "CREATE DATABASE integrador;"
```

Paso 3 — Configurar conexión

Crear `src/BackendIntegrador.Api/appsettings.Development.local.json` :

```
{
  "ConnectionStrings": {
    "DefaultConnection": "Host=localhost;Port=5432;Database=integrador;Username=postgres;Password=tupassword"
  },
  "JwtSettings": {
    "SecretKey": "ClaveSecretaSuperSeguraDe32Caracteres0Mas!",
    "Issuer": "BackendIntegrador",
    "Audience": "BackendIntegradorApp",
    "ExpirationMinutes": 60
  },
  "SeedData__Enabled": true
}
```

Paso 4 — Ejecutar con seed

```
dotnet run --project src/BackendIntegrador.Api --seed
```

Esto ejecuta migraciones automáticas y crea datos demo (4 usuarios con contraseña `Secret123!`).

Paso 5 — Verificar

```
curl http://localhost:5111/api/auth/login -H "Content-Type: application/json" -d "{\"email\":\"admin@example.com\",\"password\":\"Secret123!\"}"
```

1.3 Frontend web (Next.js 16)

Paso 1 — Clonar

```
git clone https://github.com/anomalyco/ProyectoIntegrador.git
cd ProyectoIntegrador
```

Paso 2 — Instalar dependencias

```
npm install
```

Paso 3 — Configurar variables de entorno

Crear `apps/web/.env` :

```
NEXT_PUBLIC_API_URL=http://localhost:5111
```

Paso 4 — Iniciar servidor de desarrollo

```
npm run dev:web
```

Disponible en `http://localhost:3000` .

1.4 Frontend mobile (Expo 54)

Paso 1 — Instalar dependencias

```
npm install
```

Paso 2 — Configurar variables de entorno

Crear `apps/mobile/.env` :

```
EXPO_PUBLIC_API_URL=http://192.168.x.x:5111/api
```

Usar la IP local de la computadora donde corre el backend (no `localhost`).

Paso 3 — Iniciar Expo

```
npm run dev:mobile
```

Escanea el QR con Expo Go en tu teléfono.

1.5 Ejecución con Expo Go

La aplicación mobile se ejecuta completamente en **Expo Go** sin necesidad de EAS Build para desarrollo. Las características soportadas son:

Funcionalidad	Expo Go	EAS Build
Login / Registro	✓	✓
Dashboard productor y centro	✓	✓
Mapa de riesgo regional	✓	✓
QR scanner (cámara)	✓	✓
Gemelo digital (gráficos, clima, alertas)	✓	✓
Chat con IA	✓	✓
Notificaciones push	✗	✓

Importante: Expo Go requiere que el teléfono y la computadora estén en la misma red WiFi. El backend debe estar accesible desde la IP local.

2. Estructura de directorios

2.1 Monorepo — ProyectoIntegrador

```

ProyectoIntegrador/
├─ package.json           # npm workspaces (apps/*, packages/*)
├─ tsconfig.base.json     # Path aliases @proyectointegrador/*
├─ app.json               # Expo EAS config
├─ eas.json               # EAS Build profiles
├─ AGENTS.md              # Instrucciones para opencode
├─ manual-instalacion.html
├─ manual-mantenimiento.html
├─ manual-tecnico-desarrollo.html
├─
├─ packages/              # Librerías compartidas
│  └─ domain/             # @proyectointegrador/domain (zero deps)
│     └─ index.ts
│     └─ entities/User.ts
│     └─ value-objects/Email.ts
│     └─ interfaces/IUserRepository.ts, IAuthRepository.ts
│  └─ application/        # @proyectointegrador/application
│     └─ index.ts
│     └─ use-cases/       # LoginUseCase, RegisterUseCase, etc.
│     └─ dtos/            # DTOs + Zod schemas
│     └─ hooks/           # useDebounce, usePagination
│  └─ shared-infra/       # @proyectointegrador/shared-infra
│     └─ index.ts
│     └─ http/HttpClient.ts
│     └─ validators/validators.ts
│  └─ design-tokens/      # Colores, spacing, tipografía
│     └─ colors.ts, spacing.ts, typography.ts
│  └─ assets/             # Imágenes (CowHappy, CowSad, etc.)
│     └─ images/*.png
├─
├─ apps/
│  └─ mobile/             # React Native / Expo
│     └─ package.json
│     └─ tsconfig.json
│     └─ app.json
│     └─ metro.config.js
│     └─ src/
│        └─ app/          # Expo Router (file-based)
│           └─ _layout.tsx
│           └─ login.tsx, register.tsx, ...
│           └─ gemelo.tsx, riesgo-regional.tsx
│           └─ qr-scanner.tsx
│        └─ components/   # ChatModal, ResponseModal, etc.
│        └─ infrastructure/ # API clients
│        └─ providers/    # DependencyProvider
│  └─ web/                # Next.js 16
│     └─ package.json
│     └─ tsconfig.json
│     └─ next.config.ts
│     └─ postcss.config.mjs
│     └─ src/
│        └─ app/          # App Router
│           └─ layout.tsx
│           └─ page.tsx
│           └─ login/page.tsx, register/page.tsx, ...
│           └─ gemelo/page.tsx
│           └─ riesgo-regional/page.tsx
│        └─ components/   # AppLayout, ChatModal, etc.
│        └─ infrastructure/ # API clients
│        └─ providers/    # DependencyProvider

```

2.2 Backend — backend_integrador

```

backend_integrador/
├── BackendIntegrador.sln
├── Dockerfile
├── docker-compose.yml
├── .github/workflows/test_and_build.yml
├── src/
│   ├── BackendIntegrador.Domain/           # Entidades (23)
│   │   └── Entities/
│   │       ├── Usuario.cs, Rol.cs, UsuarioRol.cs
│   │       ├── CentroAcopio.cs, Finca.cs, Productor.cs
│   │       ├── Ordeno.cs, Lote.cs, Transporte.cs
│   │       ├── Muestra.cs, AnalisisCalidad.cs
│   │       ├── ParametroCalidad.cs, ResultadoParametro.cs
│   │       ├── FincaGemeloEstado.cs, LecturaClimatica.cs
│   │       ├── PrediccionGemelo.cs, AlertaGemelo.cs
│   │       └── DeviceToken.cs, Departamento.cs, Municipio.cs
│   ├── BackendIntegrador.Application/      # Abstracciones + DTOs
│   │   ├── Abstractions/                  # IAuthenticationService, IGemeloDigitalServices, etc.
│   │   ├── Dtos/                          # EntityDtos.cs, GemeloDigitalDtos.cs, etc.
│   │   └── Common/                        # JwtSettings, GemeloConstants, validators
│   ├── BackendIntegrador.Infrastructure/    # Implementaciones
│   │   ├── Persistence/                  # AppDbContext, EfRepository
│   │   ├── Migrations/                  # 7 migraciones EF Core
│   │   ├── Services/
│   │   │   ├── AuthenticationService.cs
│   │   │   ├── PushNotificationService.cs
│   │   │   ├── EmailService.cs
│   │   │   ├── EntityCrudServices.cs    # 12 CRUD services
│   │   │   └── GemeloDigital/           # FincaGemeloService, CentroAcopioGemeloService
│   │   │                                           # OpenMeteoClimateProvider, HeuristicMilkQualityPredictor
│   │   │                                           # AlertaGemeloEvaluator, GemeloClimateMath
│   │   └── Seeding/DatabaseSeeder.cs
│   └── DependencyInjection.cs
│   ├── BackendIntegrador.Api/             # Punto de entrada
│   │   ├── Program.cs
│   │   ├── Controllers/                  # 20 controladores
│   │   ├── Filters/                      # ApiResponseFilter
│   │   ├── Middleware/                   # ExceptionHandlingMiddleware
│   │   └── Common/                       # ApiResponse, UtcDateTimeConverter
│   └── test/
│       ├── BackendIntegrador.Tests/      # Unit tests (xUnit + Moq)
│       └── BackendIntegrador.IntegrationTests/ # Integration tests (Testcontainers)

```

3. Arquitectura del código fuente

El proyecto sigue una arquitectura de **Clean Architecture** tanto en el backend como en el frontend, con dependencias unidireccionales desde la capa de presentación hacia el dominio.

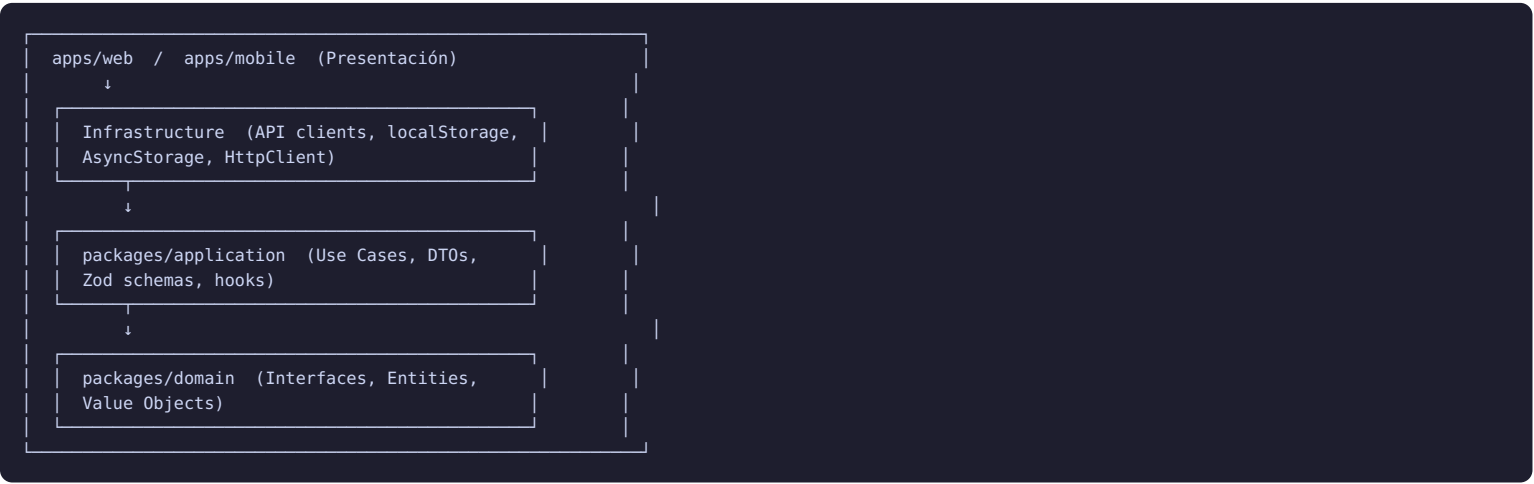
3.1 Backend — Clean Architecture en .NET 8



Principios aplicados:

- **Dependency Inversion:** Infrastructure implementa interfaces definidas en Application
- **Repository Pattern:** `IRepository<T>` → `EfRepository<T>` para abstraer EF Core
- **CRUD Service Pattern:** `IntKeyCrudServiceBase` para operaciones CRUD genéricas
- **Controller Base:** `IntKeyCrudControllerBase` provee endpoints CRUD automáticos
- **Facade Pattern:** `UsuarioFacadeService` para registro complejo multi-entidad
- **Strategy Pattern:** Gemelo Digital usa interfaces plugueables (`IClimateDataProvider` , `IMilkQualityPredictor`)
- **Filter Pipeline:** `ApiResponseFilter` envuelve respuestas en `ApiResponse<T>`
- **Exception Middleware:** Manejo global de excepciones no controladas

3.2 Frontend — Clean Architecture en TypeScript



Inyección de dependencias:

- Cada plataforma tiene su propio `DependencyProvider.tsx` (React Context)
- Los repositorios se instancian como singletons a nivel de módulo
- Los use cases reciben repositorios por constructor
- El hook `useDependencies()` expone todos los casos de uso

3.3 Flujo de datos típico



4. Convenciones de código y estándares

4.1 Backend (C# / .NET)

Convención	Estándar
Nombres de clases/métodos	PascalCase
Parámetros/variables locales	camelCase
Propiedades públicas	PascalCase
Campos privados	_camelCase (underscore prefix)
Interfaces	Prefijo I (ej. IUserRepository)
Async pattern	Sufijo Async en métodos asíncronos
DTOs	Sufijo Dto (ej. LoginDto)
Atributos de autorización	<code>[AuthorizeRole("rol")]</code>
Controladores	Extienden <code>IntKeyCrudControllerBase</code> para CRUD estándar
Métodos de endpoint	Nombres descriptivos: <code>GetAll</code> , <code>GetById</code> , <code>Create</code> , <code>Update</code> , <code>Delete</code>

4.2 Frontend (TypeScript / React)

Convención	Estándar

Nombres de archivos	camelCase (login.tsx, userRepository.ts)
Nombres de componentes	PascalCase (ChatModal.tsx, AppLayout.tsx)
Nombres de funciones/variables	camelCase
Interfaces/Types	PascalCase (UserDTO, AuthResponse)
Archivos de ruta Expo Router	kebab-case (forgot-password.tsx, qr-scanner.tsx)
Directorios Next.js App Router	kebab-case (forgot-password/page.tsx)
Hooks personalizados	Prefijo use (useDebounce, usePagination)
APIs clients	nombreApi.ts (dashboardApi.ts, gemeloApi.ts)
Zod schemas	nombreSchema.ts (loginSchema.ts)

4.3 Estándares generales

- **Commits en español:** mensajes descriptivos en presente imperativo
- **No comentarios en código:** el código debe ser auto-documentado
- **Seguridad:** no exponer secrets/keys en repositorio
- **Typescript estricto:** tipos explícitos en interfaces públicas
- **Zod para validación:** schemas en application layer, no en presentación
- **Manejo de errores:** `unwrapResponse()` para estandarizar respuestas del backend

5. Descripción de módulos y componentes

5.1 Paquetes compartidos (packages/)

@proyectointegrador/domain

Entidades y contratos puros, sin dependencias externas.

Archivo	Exporta
entities/User.ts	Interfaz <code>User</code> (id, name, email, createdAt)
value-objects/Email.ts	Clase <code>Email</code> con factory y validación
interfaces/IUserRepository.ts	<code>findById</code> , <code>findAll</code> , <code>save</code> , <code>delete</code>
interfaces/IAuthRepository.ts	<code>login</code> , <code>register</code> , <code>forgotPassword</code> , <code>verifyResetCode</code> , <code>resetPassword</code> , <code>logout</code> , <code>getSession</code>

@proyectointegrador/application

Casos de uso, DTOs, esquemas de validación Zod y hooks puros.

Archivo	Exporta
use-cases/LoginUseCase.ts	Ejecuta login, recibe <code>IAuthRepository</code>
use-cases/RegisterUseCase.ts	Ejecuta registro multi-rol
use-cases/GetUserUseCase.ts	Obtener usuario por ID
use-cases/GetAllUsersUseCase.ts	Listar todos los usuarios
use-cases/ForgotPasswordUseCase.ts	Iniciar recuperación de contraseña
use-cases/VerifyResetCodeUseCase.ts	Verificar código de 5 dígitos
use-cases/ResetPasswordUseCase.ts	Restablecer contraseña
dtos/auth/AuthResponseDto.ts	<code>AuthResponseDto</code> , <code>UsuarioDto</code> , <code>LoginDto</code>
dtos/auth/loginSchema.ts	Esquema Zod para login
hooks/useDebounce.ts	Hook genérico de debounce
hooks/usePagination.ts	Hook de paginación local

@proyectointegrador/shared-infra

Infraestructura compartida: HTTP client y validadores.

Archivo	Métodos
http/HttpClient.ts	get<T> , post<T> , put<T> , patch<T> , delete<T> — setea Content-Type y Authorization Bearer
validators/validators.ts	isValidEmail , isEmpty , minLength

@proyectointegrador/design-tokens

Tokens de diseño compartidos entre web y mobile.

Archivo	Exporta
colors.ts	primary (#534AB7) , danger , warning , success , más sub-objetos text , background , border
spacing.ts	Escala xs(4) a xxl(48)
typography.ts	Font sizes (xs-xxl), weights, line heights

@proyectointegrador/assets

Imágenes (CowHappy, CowSad, etc.) exportadas como assetPaths .

5.2 Frontend mobile (apps/mobile)

Pantallas (Expo Router — src/app/)

Archivo	Ruta	Descripción
_layout.tsx	/	Root layout: SafeAreaProvider, DependencyProvider, Stack navigator, listener de push notifications
login.tsx	/login	Formulario con react-hook-form + Zod, redirect según rol
register.tsx	/register	Registro multi-paso (3 roles) con geocoding y API Colombia
dashboard.tsx	/dashboard	Dashboard productor: calendario, análisis, notificaciones, chat
dashboard-centro.tsx	/dashboard-centro	Dashboard centro: bottom bar dinámica (QR para trabajador, trabajadores para admin)
admin.tsx	/admin	Gestión de usuarios por rol
lotes.tsx	/lotes	Lista de lotes con QR, compartir vía expo-sharing
ordenos.tsx	/ordenos	Registro de ordeño + QR + creación de lotes
transportes.tsx	/transportes	Lista de transportes, completar transporte
analisis.tsx	/analisis	Flujo de análisis de calidad: seleccionar lote → muestras → resultados
parametros.tsx	/parametros	CRUD de parámetros de calidad del centro
gemelo.tsx	/gemelo	Gemelo digital: gauge SVG, gráfico de clima (gifted-charts), predicciones, alertas
riesgo-regional.tsx	/riesgo-regional	Mapa nativo con Circle + Marker coloreados, Callout con detalle, scroll de fincas
qr-scanner.tsx	/qr-scanner	Escáner QR con expo-camera

Infraestructura (src/infrastructure/)

Archivo	Función principal
AuthRepositoryMobile.ts	Implementa IAuthRepository con HttpClient + AsyncStorage
UserRepositoryMobile.ts	Implementa IUserRepository con HttpClient + cache
dashboardApi.ts	getMiPerfil() , getAnalisisPorFinca()
gemeloApi.ts	DTOs gemelo + getEstado , getClima , getPredicciones , getAlertas , sincronizar , marcarLeida , getRiesgoRegional
ordenosApi.ts	CRUD ordenos, lotes, transportes + completarTransporte
analisisApi.ts	CRUD muestras, analisis, resultados
parametrosApi.ts	CRUD parámetros de calidad
chatApi.ts	Chat con backend de IA (puerto 8000)
colombiaApi.ts	Departamentos/ciudades desde api-colombia.com
geocode.ts	Nominatim geocoding inverso

ubicacionApi.ts	CRUD local de departamentos/municipios
notificacionesApi.ts	Registro/eliminación de push token

Componentes (src/components/)

- **ChatModal.tsx** — Asistente IA integrado
- **ResponseModal.tsx** — Modal de éxito/error con imágenes de vaca
- **ParametroCircularChart.tsx** — Gauge SVG circular para parámetros
- **TrabajadoresModal.tsx** — Lista de trabajadores del centro
- **UserDetailModal.tsx** — Detalle/edición de usuario para admin

5.3 Frontend web (apps/web)

Pantallas (Next.js App Router — src/app/)

Ruta	Archivo	Descripción
/	page.tsx	Redirecciona a /login
/login	login/page.tsx	Formulario login con react-hook-form + Zod. Redirect según rol del usuario.
/register	register/page.tsx	Registro multi-rol (Productor, Centro de Acopio, Trabajador). Incluye selección de departamento/municipio vía API Colombia y geocoding Nominatim.
/forgot-password	forgot-password/page.tsx	Ingreso de email para recuperación de contraseña.
/verify-code	verify-code/page.tsx	Ingreso de código de 5 dígitos enviado por email.
/reset-password	reset-password/page.tsx	Formulario de nueva contraseña + confirmación.
/dashboard	dashboard/page.tsx	Dashboard productor: calendario de eventos, gráficos de análisis de calidad (Recharts), lista de notificaciones, chat con IA. Envuelto en AppLayout.
/dashboard-centro	dashboard-centro/page.tsx	Dashboard centro de acopio: acceso a análisis, transportes, parámetros, trabajadores, gemelo y riesgo regional. Sidebar (AppLayout) en vez de bottom bar.
/admin	admin/page.tsx	Gestión de usuarios: listar por rol, editar datos personales (departamento/municipio con dropdowns), activar/desactivar usuarios.
/ordenos	ordenos/page.tsx	Registro de ordeños: formulario con fecha, hora, volumen. Generación de código QR del ordeño. Creación de lotes asociados.
/lotes	lotes/page.tsx	Lista de lotes con estados (Abierto / En tránsito / Entregado). QR generado con qrcode.react (sin expo-sharing). Badge de estado por lote.
/transportes	transportes/page.tsx	Lista de transportes. Botón "Completar" para registrar hora de llegada. Filtro por estado.
/analisis	analisis/page.tsx	Flujo de análisis de calidad: seleccionar lote → ver/crear muestras → ingresar resultados por parámetro. Gráfico circular de parámetros.
/parametros	parametros/page.tsx	CRUD de parámetros de calidad del centro (nombre, unidad, valores mín/máx, orden).
/gemelo	gemelo/page.tsx	Gemelo digital: gauge SVG circular con score de riesgo, gráfico de clima (Recharts LineChart con temp/THI), predicciones 7d (Recharts BarChart), lista de alertas. Acepta query params <code>fincaId</code> / <code>fincaNombre</code> para drill-down desde riesgo-regional.
/riesgo-regional	riesgo-regional/page.tsx	Mapa Leaflet con CircleMarker coloreados (rojo ≥ 60, amarillo ≥ 30, verde < 30), tamaño proporcional al score. Popup con detalle de finca + enlace a gemelo. Stats bar (promedio, total fincas, alertas activas). Chips de fincas debajo del mapa.

Infraestructura (src/infrastructure/)

Archivo	Función principal
AuthRepositoryWeb.ts	Implementa IAuthRepository con HttpClient + localStorage (token, usuario)
UserRepositoryWeb.ts	Implementa IUserRepository con HttpClient
dashboardApi.ts	<code>getMiPerfil()</code> , <code>getAnalisisPorFinca()</code>
gemeloApi.ts	Mismos DTOs y funciones que mobile: <code>getEstado</code> , <code>getClima</code> , <code>getPredicciones</code> , <code>getAlertas</code> , <code>sincronizar</code> , <code>marcarLeida</code> , <code>getRiesgoRegional</code>
ordenosApi.ts	CRUD ordenos, lotes, transportes + <code>completarTransporte</code>
analisisApi.ts	CRUD muestras, analisis, resultados

parametrosApi.ts	CRUD parámetros de calidad
chatApi.ts	Chat con backend de IA (puerto 8000)
colombiaApi.ts	Departamentos/ciudades desde api-colombia.com
geocode.ts	Nominatim geocoding inverso
ubicacionApi.ts	CRUD local de departamentos/municipios

La web no tiene `notificacionesApi.ts` — las notificaciones push son solo mobile.

Componentes (src/components/)

- **AppLayout.tsx** — Sidebar de navegación con ítems según rol. Array `topNav` define rutas y permisos. Muestra nombre del usuario (fetch a `/api/usuarios/public/{id}`). Botón de logout limpia `localStorage` y redirige a `/login`.
- **ChatModal.tsx** — Asistente IA integrado en dashboard. Input de texto + scroll de mensajes.
- **ResponseModal.tsx** — Modal de éxito/error usando Tailwind CSS + `lucide-react`.
- **ParametroCircularChart.tsx** — Gauge SVG circular que muestra valor vs rango óptimo del parámetro.

Layout (src/app/layout.tsx)

- **Server component** raíz: define `<html lang="es">` y las fuentes de Next.js
- Importa `globals.css` que usa `@import "tailwindcss"` (Tailwind v4)
- Envuelve `<DependencyProvider>` (marcado como `'use client'`)
- Todas las páginas dentro de `AppLayout` usan Tailwind CSS v4 para diseño responsive

5.4 Backend — capas y servicios

Dominio (23 entidades)

Entidades principales del negocio:

- **Usuario / Rol / UsuarioRol** — Autenticación y autorización multi-rol
- **CentroAcopio / Productor / Finca / Trabajador** — Entidades del negocio
- **Ordeno / Lote / Transporte / RecepcionAcopio** — Trazabilidad de la leche
- **Muestra / AnalisisCalidad / ParametroCalidad / ResultadoParametro** — Análisis de calidad
- **FincaGemeloEstado / LecturaClimatica / PrediccionGemelo / AlertaGemelo** — Gemelo digital
- **DeviceToken** — Notificaciones push
- **Departamento / Municipio / TipoDocumento** — Catálogos

Aplicación (Abstracciones + DTOs)

- **IAuthenticationService** — Login, cambio/reset de contraseña
- **IGemeloDigitalServices** — 6 interfaces: `IClimateDataProvider`, `IMilkQualityPredictor`, `IALertaGemeloEvaluator`, `IFincaGemeloAuthorizationService`, `IFincaGemeloService`, `ICentroAcopioGemeloService`
- **ICrudService<TRead, TCreate, TUpdate>** — CRUD genérico
- **IUsuarioFacadeService** — Provisionamiento complejo de usuarios
- **IService / INotificationService** — Email y push

Infraestructura — Servicios clave

Servicio	Responsabilidad
AuthenticationService	Login con BCrypt + JWT, forgot/reset password con MailKit SMTP
PushNotificationService	Envío vía Expo Push API, limpieza automática de tokens inválidos
FincaGemeloService	Orquestador del gemelo digital: sincronización, predicciones, alertas, score de riesgo
CentroAcopioGemeloService	Riesgo regional: agrega scores de fincas por centro
OpenMeteoClimateProvider	Cliente HTTP para Open-Meteo API (histórico + pronóstico)
HeuristicMilkQualityPredictor	Predicciones basadas en heurísticas (THI, pH, volumen)
AlertaGemeloEvaluator	Evaluación y deduplicación de alertas (ola de calor, caída de volumen)
UsuarioFacadeService	Registro transaccional de usuario + roles + datos específicos
DatabaseSeeder	Seed de datos demo (departamentos, fincas, usuarios, lecturas climáticas)

Program.cs — Pipeline

- 1. `EnableLegacyTimestampBehavior = true`
- 2. Carga de configuración: `appsettings.{env}.local.json` + `DATABASE_URL` fallback
- 3. Binding de settings: `JwtSettings` , `EmailSettings` , `OpenMeteoSettings` , `GemeloDigitalSettings`
- 4. Autenticación JWT Bearer con `ClockSkew = TimeSpan.Zero`
- 5. Controladores con filtro global `ApiResponseFilter`
- 6. Swagger/OpenAPI con definición de seguridad Bearer
- 7. CORS: AllowAnyOrigin (desarrollo)
- 8. DI de infraestructura via `services.AddInfrastructure()`
- 9. `db.Database.Migrate()` al iniciar
- 10. Seed opcional via `SeedData.Enabled` o `--seed`
- 11. Middleware: ExceptionHandling → CORS → HTTPS → Auth → Controllers

6. APIs y endpoints documentados

El backend expone Swagger UI en `http://localhost:5111/swagger` en entornos Development o Docker.

6.1 Endpoints públicos (sin autenticación)

Método	Ruta	Descripción
POST	<code>/api/auth/login</code>	Login con email/password, devuelve JWT
POST	<code>/api/auth/forgot-password</code>	Envía código de 5 dígitos al email
POST	<code>/api/auth/verify-reset-code</code>	Verifica código de recuperación
POST	<code>/api/auth/reset-password</code>	Restablece contraseña con código verificado
POST	<code>/api/usuarios</code>	Crear usuario (registro público)
GET	<code>/api/usuarios/public/{id}</code>	Obtener datos públicos de usuario
GET	<code>/api/centros-acopio</code>	Listar centros de acopio
GET	<code>/api/centros-acopio/{id}/trabajadores</code>	Listar trabajadores de un centro
GET	<code>/api/lotes/por-centro/{id}</code>	Lotes por centro (con transporte y finca)
GET	<code>/api/lotes/por-finca/{id}</code>	Lotes por finca
GET	<code>/api/ordenos/por-finca/{id}</code>	Ordeños por finca
GET	<code>/api/transportes/por-centro/{id}</code>	Transportes por centro
GET	<code>/api/analisis-calidad/por-finca/{id}</code>	Análisis por finca con resultados
GET	<code>/api/parametros-calidad/centro/{id}</code>	Parámetros por centro
GET	<code>/api/departamentos</code>	Listar departamentos
GET	<code>/api/municipios</code>	Listar municipios
POST	<code>/api/notificaciones/registrars-token</code>	Registrar token de push notification
DELETE	<code>/api/notificaciones/token</code>	Eliminar token de dispositivo

6.2 Endpoints autenticados (requieren JWT)

Método	Ruta	Descripción
GET/POST/PUT/DELETE	<code>/api/usuarios[/]{id}]</code>	CRUD usuarios
GET/POST/PUT/DELETE	<code>/api/centros-acopio[/]{id}]</code>	CRUD centros de acopio
GET/POST/PUT/DELETE	<code>/api/fincas[/]{id}]</code>	CRUD fincas
GET/POST/PUT/DELETE	<code>/api/productores[/]{id}]</code>	CRUD productores
GET/POST/PUT/DELETE	<code>/api/lotes[/]{id}]</code>	CRUD lotes (auto-genera código LT-*)
GET/POST/PUT/DELETE	<code>/api/ordenos[/]{id}]</code>	CRUD ordeños (auto-genera código ORD-*)

GET/POST/PUT/DELETE	/api/transportes/{id}	CRUD transportes
POST	/api/transportes/{id}/completar	Marca transporte como entregado, envía push
GET/POST/PUT/DELETE	/api/muestras/{id}	CRUD muestras
GET/POST/PUT/DELETE	/api/analisis-calidad/{id}	CRUD análisis (al crear, envía push al productor)
GET/POST/PUT/DELETE	/api/parametros-calidad/{id}	CRUD parámetros
GET/POST/PUT/DELETE	/api/resultados-parametro/{analisisId}/{parametroId}	CRUD resultados (PK compuesta)
GET/POST/PUT/DELETE	/api/recepciones-acopio/{id}	CRUD recepciones
GET/POST/PUT/DELETE	/api/roles/{id}	CRUD roles
POST	/api/usuarios/{id}/reset-password	Admin resetea contraseña de usuario
POST	/api/auth/change-password	Cambiar contraseña del usuario actual

6.3 Endpoints del gemelo digital

Método	Ruta	Descripción
GET	/api/fincas/{fincaId}/gemelo/estado	Estado actual del gemelo (score, clima reciente, alertas)
GET	/api/fincas/{fincaId}/gemelo/clima	Lecturas climáticas históricas (rango de fechas opcional)
GET	/api/fincas/{fincaId}/gemelo/predicciones	Predicciones activas (horizonte opcional)
GET	/api/fincas/{fincaId}/gemelo/alertas	Alertas (filtro activas opcional)
POST	/api/fincas/{fincaId}/gemelo/sincronizar	Disparar sincronización del gemelo
PATCH	/api/fincas/{fincaId}/gemelo/alertas/{alertaId}/leida	Marcar alerta como leída
GET	/api/centros-acopio/{centroId}/gemelo/riesgo-regional	Riesgo regional: fincas del centro agrupadas por score

6.4 Formato de respuesta estandarizado

Todas las respuestas usan el envelope ApiResponse<T> :

```
{
  "success": true,
  "data": { ... },
  "message": null,
  "errors": null
}
```

En caso de error:

```
{
  "success": false,
  "data": null,
  "message": "Error message",
  "errors": ["Detail 1", "Detail 2"]
}
```

El frontend usa unwrapResponse() para extraer data y lanzar error si success=false .

7. Guía de testing

7.1 Tests unitarios (backend — xUnit + Moq + FluentAssertions)

Ubicación: test/BackendIntegrador.Tests/

Ejecutar

```
dotnet test test/BackendIntegrador.Tests/BackendIntegrador.Tests.csproj
```

Cobertura

Archivo	Lo que prueba
---------	---------------

<code>GemeloDigitalUnitTests.cs</code>	<code>HeuristicMilkQualityPredictor</code> (predicción de volumen, riesgo de acidificación, score global), <code>AlertaGemeloEvaluator</code> (evaluación de ola de calor y caída de volumen, deduplicación), <code>GemeloClimateMath</code> (cálculo de THI)
<code>UsuarioRoleValidatorTests.cs</code>	Validación de reglas de roles en provisioning y actualización
<code>*ControllerUnitTests.cs</code> (18 archivos)	Tests unitarios para cada controlador CRUD con mocks de servicios

Framework: xUnit v2.6.2, Moq v4.20.2, FluentAssertions v6.11.0

7.2 Tests de integración (backend — Testcontainers)

Ubicación: `test/BackendIntegrador.IntegrationTests/`

Ejecutar

```
dotnet test test/BackendIntegrador.IntegrationTests/BackendIntegrador.IntegrationTests.csproj
```

Requiere Docker Desktop corriendo (los tests levantan un contenedor PostgreSQL 16 Alpine automáticamente).

Infraestructura

- `PostgresWebApplicationFactory.cs` — Factory personalizado que reemplaza DbContext por Testcontainer PostgreSQL
- `IntegrationTestBase.cs` — Clase base con setup de JWT, seed helpers
- `JwtTokenGenerator.cs` — Genera tokens JWT de prueba
- `FakeClimateDataProvider.cs` — Stub para Open-Meteo API

Cobertura

Archivo	Lo que prueba
<code>CrudIntegrationTests.cs</code>	Todos los endpoints CRUD devuelven OK/Created
<code>GemeloDigitalIntegrationTests.cs</code>	Sincronización con/sin coordenadas
<code>UsuariosIntegrationTests.cs</code>	Flujo completo de usuarios
<code>UsuariosPorRolIntegrationTests.cs</code>	Usuarios filtrados por rol

7.3 Tests mobile (Jest + jest-expo)

Ubicación: `apps/mobile/`

Ejecutar

```
npm run test --workspace=apps/mobile
```

Configuración

```
"jest": {
  "preset": "jest-expo",
  "setupFiles": ["/.jest.setup.js"],
  "transformIgnorePatterns": [
    "node_modules/(?!(jest-)?react-native|react-native(-community)?|expo(nent)?@expo(nent)?/.*|react-navigation|@react-navigation/.*|react-na"
  ]
}
```

Dependencias: `jest`, `jest-expo`, `@testing-library/react-native`, `@testing-library/jest-native`.

Actualmente no hay archivos de test implementados en mobile. Los test files deben seguir el patrón `*.test.tsx` o `*.spec.tsx`.

7.4 Tests web

No hay configuración de testing en `apps/web` actualmente. Para agregar tests, se recomienda usar **Vitest** o **Jest** con `@testing-library/react`.

8. Proceso de CI/CD

8.1 Backend — GitHub Actions

Archivo: backend_integrador/.github/workflows/test_and_build.yaml

Evento	Branch	Acción
Push	main	Ejecuta tests unitarios + build Debug/Release
Pull Request	main	Ejecuta tests unitarios + build

Jobs

- build** (strategy matrix: Debug, Release)
 - Checkout + setup .NET 8.0 + MSBuild
 - dotnet test (unit tests)
 - msbuild /t:Restore /p:Configuration=\$env:Configuration
 - Upload artifact (compilado)

8.2 Backend — Docker

Dockerfile (multi-stage):

- Stage 1: SDK 8.0 — restore, build, publish Release
- Stage 2: ASP.NET 8.0 — runtime, EXPOSE 8080
- ENV: ASPNETCORE_URLS=http://+:8080 , ASPNETCORE_ENVIRONMENT=Production

docker-compose.yml:

```
services:
  api:
    build: .
    ports:
      - "${API_PORT:-5111}:8080"
    environment:
      - ASPNETCORE_ENVIRONMENT=Docker
      - ConnectionStrings__DefaultConnection=...
      - JwtSettings__SecretKey=...
    env_file: .env
```

8.3 Frontend web — build

```
npm run build:web
# Salida: apps/web/.next/
```

El build se despliega en un servidor Node.js (pm2, IIS, Vercel, etc.).

8.4 Frontend mobile — EAS Build

Archivo: eas.json :

```
{
  "build": {
    "development": {
      "developmentClient": true,
      "distribution": "internal"
    },
    "preview": {
      "distribution": "internal"
    },
    "production": {}
  }
}
```

```
npx eas build --platform android --profile development
npx eas build --platform android --profile production
```

8.5 Despliegue en producción (Render)

El backend se despliega como Web Service en Render con runtime Docker. Variables de entorno configuradas en el panel de Render. PostgreSQL gestionado por Render. Seed deshabilitado en producción.

9. Gestión de versiones (Git Flow)

9.1 Repositorios

Repositorio	URL	Branch principal
Backend	github.com/anomalyco/backend_integrador	main
Frontend	github.com/anomalyco/ProyectoIntegrador	Web

9.2 Estrategia de branching

Se sigue un flujo simplificado basado en Git Flow:

- **main / Web** — Rama de producción, código estable
- **feature/*** — Ramas de características (ej. `feature/riesgo-regional`)
- **fix/*** — Ramas de corrección de bugs

9.3 Convenciones de commit

- Mensajes en **español**
- Formato: presente imperativo, descriptivo
- Ejemplos:
 - `Agregar mapa de riesgo regional con markers coloreados`
 - `Corregir validación de email en registro`
 - `Actualizar dependencias de Expo a SDK 54`

9.4 Versionado semántico

No se aplica versionado semántico formal. Las versiones de los documentos referencian la versión del manual (1.0, 1.5, etc.).

10. Contribución al proyecto

10.1 Configuración inicial para nuevos desarrolladores

1. Clonar ambos repositorios
2. Seguir la [Guía de configuración del entorno](#) (sección 1)
3. Ejecutar el backend con `--seed` para tener datos demo
4. Iniciar web y mobile con los comandos correspondientes
5. Verificar login con `admin@example.com / Secret123!`

10.2 Flujo de trabajo para nuevas características

1. Crear rama `feature/nombre-caracteristica` desde `main` (backend) o `Web` (frontend)
2. Implementar siguiendo la arquitectura establecida:
 - **Backend:** DTO → Service (interfaz + impl) → Controller
 - **Frontend:** Domain interface → Application use case → API client → Screen
3. Agregar tests unitarios para nueva lógica de negocio
4. Verificar que los tests existentes pasen
5. Hacer commit con mensaje descriptivo en español
6. Abrir Pull Request contra `main` o `Web`

10.3 Reglas de contributor

- **No modificar el backend sin preguntar** al líder del proyecto
- **No modificar infraestructura frontend** (rutas, config de build) sin consultar
- **No exponer secrets** en commits, usar variables de entorno
- **Seguir las convenciones de código** de la sección 4
- **No agregar comentarios** al código; el código debe ser auto-documentado
- **Commits en español**, presente imperativo
- **Nuevos paquetes** en `packages/` deben:
 1. Agregar path alias en `tsconfig.base.json`
 2. Duplicar alias en `apps/mobile/tsconfig.json`

- 3. Agregar a `next.config.ts` en `transpilePackages`

10.4 Dependencias entre repositorios

Si cambias...	Debes actualizar
DTO del backend	API client en mobile + web (<code>gemeloApi.ts</code> , etc.)
Endpoint del backend	API client en mobile + web
Interfaz de dominio	Implementación en ambos repos + use cases
Paquete compartido	Mobile tsconfig + web next.config + tsconfig.base

10.5 Troubleshooting común

Problema	Solución
Error de conexión a BD	Verificar PostgreSQL corriendo, credenciales en <code>appsettings.Development.local.json</code>
JWT no válido	<code>SecretKey</code> debe tener ≥ 32 caracteres
CORS bloquea peticiones	Verificar <code>AllowAnyOrigin()</code> en Program.cs (solo desarrollo)
QR no escanea	Permiso de cámara en el teléfono, Expo Go actualizado
Mapa no carga en mobile	Google Maps API key puede ser necesaria en iOS. Android usa Google Maps por defecto.
Expo start falla	<code>npx expo install --fix</code> para sincronizar dependencias
Push notifications no llegan	Requiere EAS Build (no funciona en Expo Go). Verificar token registrado en tabla <code>DeviceTokens</code> .

10.6 Contacto y recursos

- **Repositorio backend:** https://github.com/jarol1590/backend_integrador
- **Repositorio frontend:** https://github.com/jarol1590/frontend_integrador
- **Swagger UI:** <http://localhost:5111/swagger> (desarrollo)
- **Open-Meteo API:** <https://open-meteo.com/> (datos climáticos, sin API key)
- **Expo Push API:** <https://docs.expo.dev/push-notifications/sending-notifications/>